

LAB 4. FORM THU THẬP THÔNG TIN VÀ UPLOAD HÌNH ẢNH

Mục tiêu bài Lab:

1. Thiết kế được Form HTML chuẩn để gửi dữ liệu và file.
2. Sử dụng đối tượng Request để trích xuất thông tin văn bản và file đính kèm.
3. Lưu trữ hình ảnh an toàn vào hệ thống Storage của Laravel.
4. Trả về thông báo thành công cho người dùng bằng Flash Session.

BƯỚC 1: TẠO CONTROLLER VÀ ĐỊNH TUYẾN (ROUTE)

Thay vì viết code trực tiếp trong Route, chúng ta sẽ tổ chức code chuẩn MVC bằng cách tạo Controller.

1. Tạo Controller:

- Mở Terminal trong VS Code và chạy lệnh:

```
php artisan make:controller ProductController
```

2. Khai báo Route:

- Mở file routes/web.php và thêm 2 định tuyến (một để hiển thị Form, một để xử lý dữ liệu Form gửi lên):

```
use App\Http\Controllers\ProductController;

// Route hiển thị Form thêm sản phẩm (GET)
Route::get('/products/create', [ProductController::class, 'create'])->
>name('products.create');

// Route xử lý dữ liệu khi người dùng bấm Submit (POST)
Route::post('/products/store', [ProductController::class, 'store'])->
>name('products.store');
```

BƯỚC 2: THIẾT KẾ GIAO DIỆN FORM (VIEW)

Tạo một thư mục mới tên là products bên trong resources/views/. Sau đó tạo file create.blade.php.

LAB 4. FORM THU THẬP THÔNG TIN VÀ UPLOAD HÌNH ẢNH

Lưu ý: Để Form có thể gửi được file hình ảnh, thẻ <form> bắt buộc phải có thuộc tính `enctype="multipart/form-data"`.

– File: resources/views/products/create.blade.php

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <title>Thêm Sản Phẩm Mới</title>
    <style>
        body { font-family: Arial, sans-serif; background-color: #f8f9fa;
padding: 50px; }
        .form-container { background: #fff; padding: 30px; border-radius:
8px; box-shadow: 0 4px 6px rgba(0,0,0,0.1); max-width: 500px; margin: 0
auto; }
        .form-group { margin-bottom: 15px; }
        .form-group label { display: block; font-weight: bold; margin-
bottom: 5px; }
        .form-group input { width: 100%; padding: 10px; border: 1px solid
#ccc; border-radius: 4px; box-sizing: border-box; }
        .btn-submit { background-color: #28a745; color: white; border:
none; padding: 10px 20px; border-radius: 4px; cursor: pointer; width: 100%;
font-size: 16px; }
        .alert-success { background-color: #d4edda; color: #155724;
padding: 15px; border-radius: 4px; margin-bottom: 20px; border: 1px solid
#c3e6cb; }
    </style>
</head>
<body>

    <div class="form-container">
        <h2>Thêm Sản Phẩm Mới</h2>

        @if (session('success'))
            <div class="alert-success">
                <strong>Thành công!</strong> {{ session('success') }}
                <br>
                
            </div>
        @endif
    </div>
```

```
@endif

<form action="{{ route('products.store') }}" method="POST"
enctype="multipart/form-data">

    @csrf

    <div class="form-group">
        <label>Tên sản phẩm:</label>
        <input type="text" name="product_name" required
placeholder="Nhập tên sản phẩm...">
    </div>

    <div class="form-group">
        <label>Giá bán (VNĐ):</label>
        <input type="number" name="product_price" required
placeholder="Ví dụ: 150000">
    </div>

    <div class="form-group">
        <label>Hình ảnh sản phẩm:</label>
        <input type="file" name="product_image" accept="image/*"
required>
    </div>

    <button type="submit" class="btn-submit">Lưu Sản Phẩm</button>
</form>
</div>

</body>
</html>
```

BƯỚC 3: XỬ LÝ LOGIC TRONG CONTROLLER

Mở file `app/Http/Controllers/ProductController.php` và viết code cho 2 hàm `create` và `store`.

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class ProductController extends Controller
```

LAB 4. FORM THU THẬP THÔNG TIN VÀ UPLOAD HÌNH ẢNH

```
{  
    // Hàm hiển thị giao diện Form  
    public function create()  
    {  
        return view('products.create');  
    }  
  
    // Hàm nhận và xử lý dữ liệu khi Submit Form  
    public function store(Request $request)  
    {  
        // 1. Lấy dữ liệu văn bản từ Request  
        $name = $request->input('product_name');  
        $price = $request->input('product_price');  
  
        // Biến chứa đường dẫn ảnh (mặc định là rỗng)  
        $imagePath = '';  
  
        // 2. Kiểm tra xem người dùng có upload file hay không  
        if ($request->hasFile('product_image')) {  
            $file = $request->file('product_image');  
  
            // 3. Lưu file vào ổ đĩa 'public', bên trong thư mục 'products'  
            // Hàm store() sẽ tự động tạo tên file ngẫu nhiên để không bị  
trùng  
            $imagePath = $file->store('products', 'public');  
        }  
  
        // 4. (Tùy chọn) Ở bước này thực tế bạn sẽ code lệnh lưu $name,  
$price, $imagePath vào Database.  
        // Do nội dung Database thuộc Chương sau nên chúng ta tạm bỏ qua.  
  
        // 5. Chuyển hướng người dùng quay lại form VÀ đính kèm thông báo  
Flash Data  
        return redirect()->route('products.create')  
            ->with('success', "Đã thêm sản phẩm: $name với giá $price VNĐ")  
            ->with('image_path', $imagePath);  
    }  
}
```

BƯỚC 4: KÍCH HOẠT STORAGE LINK (BẮT BUỘC)

Vì file ảnh đang được lưu tại `storage/app/public/products`, trình duyệt web không thể đọc được. Bạn cần tạo một cầu nối (shortcut) ra thư mục public bên ngoài.

- Mở Terminal và chạy lệnh sau (chỉ cần chạy 1 lần cho dự án):

```
php artisan storage:link
```

BƯỚC 5: NGHIỆM THU KẾT QUẢ

1. Mở trình duyệt, truy cập: `http://localhost:8000/products/create` (hoặc test domain của bạn).
2. Nhập tên sản phẩm (Ví dụ: Bàn phím cơ), nhập giá (Ví dụ: 1500000).
3. Chọn một tấm ảnh bất kỳ từ máy tính.
4. Bấm Lưu Sản Phẩm.
5. Kết quả mong đợi:
 - Trình duyệt tải lại trang cũ.
 - Xuất hiện thông báo màu xanh báo hiệu thành công.
 - Hiển thị tấm ảnh bạn vừa upload ngay bên dưới thông báo.
6. Mở lại VS Code, vào thư mục `storage/app/public/products`, bạn sẽ nhìn thấy file ảnh gốc đã được lưu an toàn tại đây với một cái tên mã hóa.